Kroger Kong OSS

Control Plane API Key Authentication

Complete Implementation, Troubleshooting Manual

Security note
This document deliberately does not contain the literal API key. Anywhere a valid key is required, a placeholder is used. The POC key that appeared in terminal/screenshots should be rotated before final production-style handover.

Contents

1. Executive Summary
2. What We Were Trying to Achieve
3. The Solution in One Picture
4. Plain-English Glossary
5. Exact Names, Objects, Paths and Ports
6. Before vs After Architecture
7. Detailed Step-by-Step Implementation
8. Kong Objects We Created
9. Kubernetes / Repository Changes
10. Akeyless and Secret Flow
11. Troubleshooting Journey and What Each Failure Taught Us
12. Final End-to-End Validation
13. Complete Command Reference
14. What Is Working vs What Is Still Manual
15. Next Phase: Automation Objective
16. Client Demo Script and Demo Commands
17. How to Explain the Design to a Non-Technical Person
18. Security, Rotation and Operational Notes
19. Handover Checklist

1. Executive Summary

The requirement was to secure access to the Kong OSS Control Plane (CP) Admin API with an API key. The final POC keeps the CP in its existing control-plane role and uses the Kong Data Plane (DP) as the traffic-handling security gate. The caller uses the protected path /admin-api. Kong applies its built-in key-auth plugin on that Route. A missing or incorrect key is rejected with HTTP 401; a valid key is accepted and the request is forwarded internally to the CP Admin API on kong-cp-admin:8001.

POC result
The working behavior was proven through the real ingress hostname: root path / -> 404, /admin-api with no key -> 401, /admin-api with a wrong key -> 401, and /admin-api with the registered valid key -> 200.

The implementation required two kinds of work: (1) Kong configuration objects created through the Kong Admin API, and (2) Kubernetes/repository changes that expose only the protected path and synchronize the API key from Akeyless into a Kubernetes Secret. The routing/authentication design is proven. The remaining engineering step is to automate creation/update of the Kong Service, Route, plugin, Consumer and credential so a fresh environment does not depend on manual curl commands.

2. What We Were Trying to Achieve

The goal can be explained without technical terminology: we wanted to put a security checkpoint in front of the Control Plane Admin API. A caller must present the correct API key before the administrative API is reached.

#	What we did	Why we did it	Result
1	Caller sends no API key.	Prove unauthenticated access is blocked.	401 Unauthorized
2	Caller sends an incorrect API key.	Prove simply sending a header is not enough; the credential must match Kong's registered key.	401 Unauthorized
3	Caller sends the valid registered API key.	Prove authorized administrative access works.	200 OK
4	Caller tries the hostname root path / without /admin-api.	Prove there is no easy unauthenticated bypass to the raw Admin API.	404 Not Found

Simple analogy
Think of the CP Admin API as a secure room. The DP proxy is the security guard, /admin-api is the controlled entrance, the key-auth plugin checks the access card, and admin-api-loopback is the internal corridor that reaches the actual room only after the check passes.

3. The Solution in One Picture

Final working request path

```
Caller / Postman
      |
      | HTTPS request to /admin-api
      | Header: x-api-key: <key>
      v
NGINX Ingress: kong-cp-ingress
      |
      | backend = kong-dp-proxy:80
      v
Kong Data Plane proxy
      |
      v
Kong Route: admin-api-loopback-route
Path: /admin-api   strip_path=true
      |
      v
Plugin: key-auth
Header checked: x-api-key
      |
      +---- missing/wrong key ----> 401 Unauthorized
      |
      +---- valid registered key
                |
                v
Kong Service: admin-api-loopback
Upstream: kong-cp-admin...svc.cluster.local:8001
                |
                v
Kong Control Plane Admin API
                |
                v
              200 OK
```

The API key itself originates in Akeyless. ExternalSecret synchronizes it into the Kubernetes Secret apim-nonprod-secrets under the key cp-api-key. During the POC, the same value was manually registered in Kong as a key-auth credential belonging to the Consumer cp-api-admin. Kong validates incoming requests against the credential stored in its own configuration/database; it does not call Akeyless for every request.

4. Plain-English Glossary

Term	Simple meaning
Control Plane (CP)	The management side of Kong. It owns configuration and exposes the Admin API. In this environment the CP stays in role=control_plane.
Data Plane (DP)	The traffic-handling side of Kong. It is designed to receive/proxy requests and apply Kong Routes/plugins.
Proxy	A middle layer. The caller talks to the proxy; the proxy applies rules and then forwards an allowed request to the real backend.
Ingress	The Kubernetes entry point that maps an external/internal hostname and path to a Kubernetes Service.
Kubernetes Service	A stable network name/port in Kubernetes that points to one or more pods. Examples: kong-dp-proxy and kong-cp-admin.
Kong Service	A Kong configuration object representing the upstream/backend destination. Ours is admin-api-loopback.

Term	Simple meaning
Kong Route	A Kong rule that decides which incoming request belongs to which Kong Service. Ours is admin-api-loopback-route with path /admin-api.
Plugin	A feature Kong applies to traffic. We use the built-in key-auth plugin to validate API keys.
Consumer	A Kong identity that owns credentials. We created cp-api-admin.
Credential	The actual key-auth secret registered against the Consumer. It must match the key presented in x-api-key.
Loopback Service	The name we used for the protected internal service that reaches the Admin API after authentication. The original classic loopback target was 127.0.0.1:8001; in this hybrid design it was corrected to the CP Kubernetes Service because proxy and Admin API live on different Kong roles.
Akeyless	The secure secret vault containing the CP API key.
ExternalSecret	The Kubernetes integration that copies the secret from Akeyless into a Kubernetes Secret.
Port-forward	A temporary local tunnel created by kubectl. We used it to test DP routing/authentication without depending on Ingress.
401	HTTP Unauthorized: authentication was missing or invalid.
404	HTTP Not Found: there is no exposed route for that path.
200	HTTP OK: the valid request successfully reached the CP Admin API.

5. Exact Names, Objects, Paths and Ports

Item	Exact value	Purpose
Namespace	apimkg-kong-oss-gcp-nonprod	Kubernetes namespace used for the CP/DP resources
CP hostname	kong-cp-east-nonprod.dev.gcp-internal.kroger.com	Real hostname used in final end-to-end tests
Ingress	kong-cp-ingress	Routes only /admin-api to the DP proxy in the final POC
DP Kubernetes Service	kong-dp-proxy	Traffic-handling backend for Ingress
DP Service port	80 -> targetPort 8000	Ingress points to service port 80; the Service forwards to Kong proxy listener 8000
CP Kubernetes Service	kong-cp-admin	Internal service for CP Admin API
CP Admin port	8001	Final upstream port reached after authentication
Kong Service	admin-api-loopback	Kong upstream object for the protected Admin API path
Final Kong Service upstream	http://kong-cp-admin.apimkg-kong-oss-gcp-nonprod.svc.cluster.local:8001	Internal DP-to-CP destination
Kong Route	admin-api-loopback-route	Protected Kong Route
Route path	/admin-api	Public/internal path on the CP hostname that must pass key-auth
Route setting	strip_path=true	Removes /admin-api before forwarding to the CP Admin API
Kong Plugin	key-auth	Built-in API key validation plugin
Header name	x-api-key	Only header location enabled for the key in the POC
Consumer	cp-api-admin	Kong identity that owns the key-auth credential
Akeyless secret path	/Platform/KPS/APM0002644/common/nonprod/kong-oss-gcp-eus-nonprod-cp-api-key	Secure source of the API key
ExternalSecret name	oss-kong-all-akeyless-secret	ExternalSecret syncing values into Kubernetes
Kubernetes Secret	apim-nonprod-secrets	Target Kubernetes Secret
Secret data key	cp-api-key	Name used inside apim-nonprod-secrets
CP role	control_plane	Kept unchanged
CP proxy setting	proxy.enabled: false	Final design does not depend on CP serving proxy traffic
DP role	data_plane	Traffic-handling role
DP proxy setting	proxy.enabled: true	Existing DP capability used by the solution

6. Before vs After Architecture

6.1 Initial idea / direct CP proxy attempt

```
Ingress / local test
      v
kong-cp-proxy:8000
      v
CP proxy listener
      v
127.0.0.1:8001 Admin API
```

This path did not work in the current hybrid CP runtime. A Kubernetes Service could exist, and KONG_PROXY_LISTEN could appear in the environment, but direct port-forwarding to CP port 8000 produced connection refused. This showed we should not redesign the CP into a traffic proxy.

6.2 Final POC design

```
Ingress
  v
kong-dp-proxy:80
  v
DP Route /admin-api + key-auth
  v
admin-api-loopback
  v
kong-cp-admin:8001
  v
CP Admin API
```

This honors the separation of responsibilities in the hybrid deployment: DP handles proxy traffic and authentication; CP remains the management plane and Admin API upstream.

7. Detailed Step-by-Step Implementation

The following sequence documents the actual POC journey. Commands that contain a secret use placeholders. The literal API key must never be written into documentation.

7.1 Define the CP hostname and establish a baseline

Command

```
$cp = "https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com"
```

Use: creates a PowerShell variable so later Admin API commands are shorter and less error-prone.

Command

```
curl.exe -k -i "$cp/"
```

Use: verifies that the existing CP Admin API was reachable before changes and gives a known baseline. At that stage, the root Admin API returned HTTP 200 and Kong configuration JSON.

7.2 Create the Kong Service: admin-api-loopback

Optional existence check used before create

```
curl.exe -k -i "$cp/services/admin-api-loopback"
```

Create command used in the original loopback POC

```
curl.exe -k -i -X POST "$cp/services" --data "name=admin-api-loopback" --data "url=http://127.0.0.1:8001"
```

Use: creates a Kong Service object representing the Admin API upstream. The first version used classic loopback localhost:8001. Later, because the request is proxied by DP while the Admin API lives on CP, this upstream was corrected to kong-cp-admin:8001.

7.3 Create the Kong Route: admin-api-loopback-route

```
curl.exe -k -i -X POST "$cp/services/admin-api-loopback/routes" --data "name=admin-api-loopback-route" --data
"paths[]=/admin-api" --data "strip_path=true"
```

Use: creates the protected URL path /admin-api and associates it with admin-api-loopback. strip_path=true means that when /admin-api/services is received, the upstream Admin API sees /services rather than /admin-api/services.

7.4 Attach the key-auth plugin to that Route

```
curl.exe -k -i -X POST "$cp/routes/admin-api-loopback-route/plugins" --data "name=key-auth" --data
"config.key_names[]=x-api-key" --data "config.key_in_header=true" --data "config.key_in_query=false" --data
"config.key_in_body=false" --data "config.hide_credentials=true"
```

Use: turns /admin-api into an authenticated Kong Route. The plugin looks only in the x-api-key header, not in the query string or request body. hide_credentials=true prevents the credential from being forwarded upstream.

7.5 Create the Consumer: cp-api-admin

Existence check

```
curl.exe -k -i "$cp/consumers/cp-api-admin"
```

Create command

```
curl.exe -k -i -X POST "$cp/consumers" --data "username=cp-api-admin"
```

Use: creates a Kong identity to own the API-key credential. key-auth credentials must belong to a Consumer.

7.6 Register the API-key credential

> **Important**
> The actual POC key is intentionally omitted. Use a secure variable/clipboard method and never echo the value.

Recommended secure pattern

```
$apiKey = (Get-Clipboard).Trim(); $body="key="+[uri]::EscapeDataString($apiKey); $body | curl.exe -k -s -o
NUL -w "%{http_code}" -X POST "$cp/consumers/cp-api-admin/key-auth" -H "Content-Type:
application/x-www-form-urlencoded" --data-binary "@-"
```

Expected create result: HTTP status 201. During the POC, the same secret value stored in Akeyless was registered as this Kong credential. Kong then compares each incoming x-api-key value against its registered credential.

7.7 Check CP and attempt the original CP proxy path

```
kubectl get pods -n apimkg-kong-oss-gcp-nonprod | findstr kong-cp

kubectl get svc -n apimkg-kong-oss-gcp-nonprod | findstr kong-cp

kubectl exec -n apimkg-kong-oss-gcp-nonprod <CP-POD> -c proxy -- printenv KONG_PROXY_LISTEN

kubectl exec -n apimkg-kong-oss-gcp-nonprod <CP-POD> -c proxy -- printenv KONG_ADMIN_LISTEN
```

Use: confirms CP runtime settings. The environment showed proxy_listen on 0.0.0.0:8000 and admin_listen on 0.0.0.0:8001, but runtime behavior still needed to be tested.

```
kubectl port-forward svc/kong-cp-proxy 18000:8000 -n apimkg-kong-oss-gcp-nonprod

curl.exe -i http://127.0.0.1:18000/admin-api/
```

Observed result: connection refused / empty reply during the CP proxy test. This was the decisive evidence that the CP path was not usable as the proxy entry point in this deployment.

7.8 Attempt an in-container test

```
kubectl exec -n apimkg-kong-oss-gcp-nonprod <CP-POD> -c proxy -- curl -i http://127.0.0.1:8000/admin-api/
```

Observed result: the Kong container did not contain curl. We did not install tools into the running container; instead we continued with Kubernetes port-forward tests from the workstation.

7.9 Confirm DP health and Services

```
kubectl get pods -n apimkg-kong-oss-gcp-nonprod | findstr kong-dp

kubectl get svc -n apimkg-kong-oss-gcp-nonprod | findstr kong-dp
```

Use: verifies the DP is Running/Ready and that kong-dp-proxy exists. This told us the DP is available to perform proxying/authentication while CP remains unchanged.

7.10 Port-forward DP and prove key-auth independently of Ingress

```
kubectl port-forward svc/kong-dp-proxy 18000:80 -n apimkg-kong-oss-gcp-nonprod
```

Use: maps local 127.0.0.1:18000 to the Kubernetes Service kong-dp-proxy port 80. This temporary tunnel lets us test Kong directly without NGINX Ingress.

No-key test

```
curl.exe -i http://127.0.0.1:18000/admin-api/
```

Observed result: HTTP 401 with “No API key found in request.” This proved the Route and key-auth plugin were present on the DP.

Wrong-key test

```
curl.exe -i http://127.0.0.1:18000/admin-api/ -H "x-api-key: wrong-key"
```

Observed result: HTTP 401 Unauthorized. This proved Kong was validating the credential, not merely checking whether the header exists.

7.11 Correct the Kong Service upstream from localhost to the CP Admin Service

```
curl.exe -k -i -X PATCH "$cp/services/admin-api-loopback" --data
"url=http://kong-cp-admin.apimkg-kong-oss-gcp-nonprod.svc.cluster.local:8001"
```

Observed result: HTTP 200. Use: changes the upstream from “port 8001 on the same DP network namespace” to the actual internal CP Admin Kubernetes Service. This is the key adaptation from a classic same-node loopback to the hybrid DP-to-CP topology.

7.12 Ensure the valid credential matches the Akeyless value

POC form used during testing - literal key omitted here

```
curl.exe -k -s -o NUL -w "%{http_code}" -X POST "$cp/consumers/cp-api-admin/key-auth" -H "Content-Type:
application/x-www-form-urlencoded" --data-urlencode "key=<VALID_KEY_FROM_AKEYLESS>"
```

Observed result when successfully created: 201. Use: registers the exact key that will be sent in x-api-key. The real key must not be pasted into documentation or shared screenshots.

7.13 Prove valid access through DP port-forward

```
curl.exe -i http://127.0.0.1:18000/admin-api/ -H "x-api-key: <VALID_KEY>"
```

Observed result: HTTP 200 and CP Admin API JSON. This completed the isolated 401 / 401 / 200 pattern before Ingress was changed.

8. Kong Objects We Created

These objects live in Kong configuration/database. They were created through the CP Admin API during the POC; they were not originally defined as Kubernetes YAML objects.

Object type	Name	Important settings	Purpose
Service	admin-api-loopback	Final upstream: http://kong-cp-admin.apimkg-kong-oss-gcp-nonprod.svc.cluster.local:8001	Where an authenticated request is sent
Route	admin-api-loopback-route	paths=/admin-api; strip_path=true	Which incoming path receives protection

Object type	Name	Important settings	Purpose
Plugin	key-auth	Attached to admin-api-loopback-route; x-api-key header only; hide_credentials=true	Validates the API key
Consumer	cp-api-admin	Kong username/identity	Owns the credential
Credential	key-auth credential for cp-api-admin	Secret value matches Akeyless key	Makes the valid x-api-key recognizable to Kong

How to look up the Route now

```
curl.exe -k -s "$cp/routes/admin-api-loopback-route"
```

How to look up the Kong Service now

```
curl.exe -k -s "$cp/services/admin-api-loopback"
```

Recommended verification: list plugins attached to the Route

```
curl.exe -k -s "$cp/routes/admin-api-loopback-route/plugins"
```

How to look up the Consumer

```
curl.exe -k -s "$cp/consumers/cp-api-admin"
```

Kong Route vs Ingress path

The Kong Route /admin-api is a Kong traffic rule. The Kubernetes Ingress also uses /admin-api, but its job is only to send that path to kong-dp-proxy. The Kong Route is where key-auth is actually applied.

9. Kubernetes / Repository Changes

9.1 cp-ingress.yaml

Final POC behavior: expose only /admin-api and send it to the existing DP proxy Service.

```
paths:
- path: /admin-api
  pathType: Prefix
  backend:
    service:
      name: kong-dp-proxy
      port:
        number: 80
```

Why: the DP is the proxy-capable component. Service port 80 maps internally to targetPort 8000. The earlier backend-name typo kongdp-dp-proxy caused NGINX 503 and was corrected to kong-dp-proxy.

9.2 nonprod-template-deploy.yaml

Add the CP API key path under externalsecrets.data so Akeyless can synchronize the value into apim-nonprod-secrets.

```
cp-api-key:
  path: /Platform/KPS/APM0002644/common/nonprod/kong-oss-gcp-eus-nonprod-cp-api-key
```

Important: this automates only Akeyless -> Kubernetes Secret. It does not register the credential in Kong by itself.

9.3 nonprod-cp-deploy.yaml

```
env:
  role: control_plane
  admin_listen: "0.0.0.0:8001"

proxy:
  enabled: false

admin:
  enabled: true
  type: ClusterIP
  http:
    enabled: true
    servicePort: 8001
    containerPort: 8001
```

API-key POC decision: do not turn the CP into the external proxy. Keep CP management behavior and Admin Service intact.

9.4 nonprod-dp-deploy.yaml

```
env:
  role: data_plane
  database: "off"

proxy:
  enabled: true
```

No special API-key-specific change was required in the DP values file for the POC. We used the DP proxy capability that was already enabled.

10. Akeyless and Secret Flow

```
Akeyless secret
/Platform/KPS/APM0002644/common/nonprod/kong-oss-gcp-eus-nonprod-cp-api-key
        |
        v
ExternalSecret
oss-kong-all-akeyless-secret
        |
        v
Kubernetes Secret
apim-nonprod-secrets
        |
        v
Data key
cp-api-key
```

The secret value should never be printed during routine verification. We verified only that the Kubernetes Secret contained a non-empty cp-api-key value.

Set kubeconfig in a new PowerShell window

```
$env:KUBECONFIG="C:\Users\ITTC2899\Downloads\cx-shared-01-dev-us-east1.yaml"
```

Confirm the kubectl identity/context is usable

```
kubectl auth whoami
```

Verify presence without decoding/printing the key

```
kubectl get secret apim-nonprod-secrets -n apimkg-kong-oss-gcp-nonprod -o jsonpath="{.data.cp-api-key}" |
Measure-Object -Character
```

In the POC this returned a non-zero character count (60), proving the Akeyless-to-Kubernetes synchronization worked.

11. Troubleshooting Journey and What Each Failure Taught Us

What we observed	What it meant	Decision / fix
CP port-forward failed with connection refused	Kubernetes could identify the CP Service/pod, but the CP was not serving proxy traffic on port 8000 in the way the POC needed.	Stop trying to make CP the proxy; use DP for traffic handling.
curl not found inside Kong container	The runtime image was minimal and did not include curl.	Do not modify the running container; test from the workstation using kubectl port-forward.
No-key and wrong-key both returned 401 through DP	This was a success, not a failure.	Confirmed Route and key-auth had synchronized to DP and were functioning.
Valid key returned 401 initially	Kong credential did not match the Akeyless value used for the test.	Register/confirm the exact credential for cp-api-admin.
Valid key then would need the correct upstream	127.0.0.1:8001 on DP is DP localhost, not CP.	Patch admin-api-loopback to kong-cp-admin...:8001.
ExternalSecret deployment showed resourceVersion error	Kubernetes manifest/update metadata issue, not an API-key content issue.	Do not treat it as authentication failure; inspect rendered manifest/update behavior.
kubectl tried localhost:8080	New PowerShell window did not have KUBECONFIG set.	Set $env:KUBECONFIG again and verify with kubectl auth whoami.
Real hostname returned NGINX 503	Request reached NGINX but backend routing was invalid/unavailable.	Inspect DP pod, endpoints, Service port mapping and Ingress backend.
Ingress describe showed kongdp-dp-proxy not found	Live Ingress referenced the wrong Service name.	Correct backend to kong-dp-proxy and redeploy.
After correction, no-key returned Kong 401	Ingress successfully reached DP and key-auth became the responder.	Proceed with wrong-key and valid-key acceptance tests.

11.1 Commands used to isolate the NGINX 503

```
kubectl get pods -n apimkg-kong-oss-gcp-nonprod | findstr kong-dp

kubectl get endpoints kong-dp-proxy -n apimkg-kong-oss-gcp-nonprod -o wide

kubectl get svc kong-dp-proxy -n apimkg-kong-oss-gcp-nonprod -o yaml

kubectl describe ingress kong-cp-ingress -n apimkg-kong-oss-gcp-nonprod

kubectl get ingress kong-cp-ingress -n apimkg-kong-oss-gcp-nonprod -o
jsonpath="{.spec.rules[0].http.paths[0].backend.service.name}"
```

These commands proved: DP was healthy, endpoints existed, Service port 80 correctly mapped to targetPort 8000, and the actual problem was the wrong live backend Service name. After redeployment, the JSONPath command returned exactly kong-dp-proxy.

12. Final End-to-End Validation

After the ingress backend was corrected, tests were performed against the real hostname. No port-forward was involved.

Test A - no API key

```
curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/
```

Expected/observed: HTTP 401 Unauthorized with a Kong message indicating no API key was found.

Test B - wrong API key

```
curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/ -H "x-api-key: wrong-key"
```

Expected/observed: HTTP 401 Unauthorized.

Test C - valid API key

```
curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/ -H "x-api-key:
<VALID_KEY>"
```

Expected/observed: HTTP 200 OK and CP Admin API JSON. This proves NGINX -> DP -> Route -> key-auth -> Kong Service -> CP Admin API is working end-to-end.

Test D - bypass/root path

```
curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/
```

Expected/observed: HTTP 404 Not Found. This proves the root hostname does not directly expose the CP Admin API through the configured Ingress path.

Acceptance evidence

No key = 401; wrong key = 401; valid key = 200; unprotected root path = 404.

13. Complete Command Reference

This section is a quick reference for someone who wants to understand what each command is doing. Secret-bearing commands use placeholders.

Command	What it is used for
`$cp = "https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com"`	Store CP hostname in a PowerShell variable.
`curl.exe -k -i "$cp/"`	Baseline call to the CP Admin API.
`curl.exe -k -i "$cp/services/admin-api-loopback"`	Check whether the Kong Service exists.
`curl.exe -k -i -X POST "$cp/services" --data "name=admin-api-loopback" --data "url=http://127.0.0.1:8001"`	Create the original loopback Kong Service.
`curl.exe -k -i -X POST "$cp/services/admin-api-loopback/routes" --data "name=admin-api-loopback-route" --data "paths[]=/admin-api" --data "strip_path=true"`	Create /admin-api Route linked to the Service.
`curl.exe -k -i -X POST "$cp/routes/admin-api-loopback-route/plugins" --data "name=key-auth" --data "config.key_names[]=x-api-key" --data "config.key_in_header=true" --data "config.key_in_query=false" --data "config.key_in_body=false" --data "config.hide_credentials=true"`	Attach key-auth to the Route.
`curl.exe -k -i "$cp/consumers/cp-api-admin"`	Check Consumer.
`curl.exe -k -i -X POST "$cp/consumers" --data "username=cp-api-admin"`	Create Consumer.
`kubectl get pods -n apimkg-kong-oss-gcp-nonprod`	See CP/DP pod health.
`kubectl get svc -n apimkg-kong-oss-gcp-nonprod`	See Kubernetes Services and ports.
`kubectl port-forward svc/kong-dp-proxy 18000:80 -n apimkg-kong-oss-gcp-nonprod`	Create local tunnel to DP proxy for isolated testing.
`curl.exe -i http://127.0.0.1:18000/admin-api/`	Test no-key behavior through DP port-forward.
`curl.exe -i http://127.0.0.1:18000/admin-api/ -H "x-api-key: wrong-key"`	Test invalid-key behavior.
`curl.exe -k -i -X PATCH "$cp/services/admin-api-loopback" --data "url=http://kong-cp-admin.apimkg-kong-oss-gcp-nonprod.svc.cluster.local:8001"`	Point authenticated traffic to actual CP Admin Service.
`kubectl get endpoints kong-dp-proxy -n apimkg-kong-oss-gcp-nonprod -o wide`	Check whether DP Service has ready backend endpoints.
`kubectl get svc kong-dp-proxy -n apimkg-kong-oss-gcp-nonprod -o yaml`	Check Service port -> targetPort mapping.
`kubectl describe ingress kong-cp-ingress -n apimkg-kong-oss-gcp-nonprod`	Inspect live Ingress rule, backend and events.
`kubectl get ingress kong-cp-ingress -n apimkg-kong-oss-gcp-nonprod -o jsonpath="{.spec.rules[0].http.paths[0].backend.service.name}"`	Print the exact live backend Service name.
`kubectl get secret apim-nonprod-secrets -n apimkg-kong-oss-gcp-nonprod -o jsonpath="{.data.cp-api-key}" | Measure-Object -Character`	Check that cp-api-key exists without printing the value.

Command	What it is used for
`curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/`	Final no-key ingress test.
`curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/ -H "x-api-key: wrong-key"`	Final wrong-key ingress test.
`curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/admin-api/ -H "x-api-key: <VALID_KEY>"`	Final valid-key ingress test.
`curl.exe -k -i https://kong-cp-east-nonprod.dev.gcp-internal.kroger.com/`	Final bypass/root-path test.

14. What Is Working vs What Is Still Manual

Area	Current status	Comment
Ingress /admin-api -> DP	Working	Live hostname reaches kong-dp-proxy.
key-auth no-key rejection	Working	401.
key-auth wrong-key rejection	Working	401.
valid key access	Working	200 to CP Admin API.
Root bypass	Working as intended	404.
Akeyless -> Kubernetes Secret	Working	cp-api-key present in apim-nonprod-secrets.
Kong Service creation	Manual in current POC	Created through CP Admin API.
Kong Route creation	Manual in current POC	Created through CP Admin API.
key-auth plugin creation	Manual in current POC	Created through CP Admin API.
Consumer creation	Manual in current POC	Created through CP Admin API.
Kong credential registration / rotation	Manual in current POC	Needs automation to read cp-api-key securely and create/update credential.

15. Next Phase: Automation Objective

Automation is not intended to change the working architecture. Its only purpose is to replace the manual Kong Admin API setup with a repeatable deployment-time step.

```
Akeyless
  v
ExternalSecret
  v
apim-nonprod-secrets / cp-api-key
  v
Automation / bootstrap step
  +--> ensure admin-api-loopback exists and points to kong-cp-admin:8001
  +--> ensure admin-api-loopback-route exists with /admin-api
  +--> ensure key-auth plugin is attached
  +--> ensure cp-api-admin Consumer exists
  +--> ensure key-auth credential matches cp-api-key
  v
Ready for 401 / 401 / 200 tests
```

The automation should be idempotent: running it multiple times should create missing objects or update existing ones rather than fail due to duplicates. It must never print the secret into logs. It should also support key rotation by replacing/updating the Kong credential when the Akeyless value changes.

Why automation matters

The POC works because we manually created Kong objects. A completely fresh environment would not automatically recreate those objects today. Automation closes that reproducibility gap.

Final handover statement

The POC is proven end-to-end. The protected /admin-api path is reachable through Ingress and DP, key-auth correctly enforces the API key, authenticated traffic reaches the CP Admin API, and the raw hostname root is not an Admin API bypass. The next engineering task is automation of the Kong configuration/credential bootstrap, not redesign of the working traffic path.